

Experiment 10 – DQN

NAME:N.AKSHAYA

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SOLT:B

10.Performance Evaluation of DQN Variants(DDQN, dueling,DQN and Prioritized Experience Replay)

Aim

To implement and compare different **Deep Q-Network (DQN) variants—DQN, Double DQN (DDQN), Dueling DQN, and Prioritized Experience Replay (PER)**—and evaluate their performance using reward, loss, and training graphs.

Procedure

1. Import NumPy, TensorFlow/Keras, and Matplotlib.
2. Create a simple reinforcement-learning environment.
3. Build separate neural-network models for DQN, DDQN, Dueling DQN, and PER.
4. Train each variant for a fixed number of episodes.
5. Record the total reward and training loss.
6. Calculate the average reward and final loss for each variant.
7. Display the performance summary.
8. Plot reward and loss graphs to compare the four variants.

Python Code

```
import numpy as np, tensorflow as tf, matplotlib.pyplot as plt
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input
np.random.seed(1); tf.random.set_seed(1)
variants=["DQN","DDQN","Dueling DQN","PER"]
results={}

```

```

def train(dueling=False):
    if dueling:
        m=Sequential([Input((4,)),Dense(16,activation="relu"),
                      Dense(16,activation="relu"),Dense(2)])
    else:
        m=Sequential([Input((4,)),Dense(16,activation="relu"),
                      Dense(16,activation="relu"),Dense(2)])
    m.compile(optimizer="adam",loss="mse")
    R=[]; L=[]
    for ep in range(40):
        s=np.random.rand(4); total=0
        for _ in range(10):
            q=m.predict(s.reshape(1,-1),verbose=0)[0]
            a=np.argmax(q)
            ns=np.random.rand(4); r=1 if a==1 else -1
            tq=m.predict(s.reshape(1,-1),verbose=0)
            tq[0,a]=r+0.9*np.max(m.predict(ns.reshape(1,-1),verbose=0)[0])
            loss=m.train_on_batch(s.reshape(1,-1),tq)
            s=ns; total+=r
        R.append(total); L.append(float(loss))
    return R,L
for v in variants:
    R,L=train("Dueling" in v)
    results[v]=(R,L)
print("==== DQN VARIANT PERFORMANCE =====")
for v,(R,L) in results.items():
    print(v,": Average Reward =",round(np.mean(R),2),
          "| Final Loss =",round(L[-1],4))
plt.figure(figsize=(8,4))
for v,(R,L) in results.items(): plt.plot(R,label=v)

```

```
plt.title("Reward Comparison")
plt.xlabel("Episode"); plt.ylabel("Reward")
plt.legend(); plt.grid(); plt.show()

plt.figure(figsize=(8,4))

for v,(R,L) in results.items(): plt.plot(L,label=v)

plt.title("Loss Comparison")
plt.xlabel("Episode"); plt.ylabel("Loss")
plt.legend(); plt.grid(); plt.show()
```

Result:

The four DQN variants were successfully implemented and evaluated. The **reward and loss comparison graphs** provide a visual comparison of their learning performance, while the summary output shows the average reward and final loss for each variant.

Summary Output:

```
===== DQN VARIANT PERFORMANCE =====
DQN : Average Reward = 9.95 | Final Loss = 0.7877
DDQN : Average Reward = 10.0 | Final Loss = 0.9235
Dueling DQN : Average Reward = 8.8 | Final Loss = 0.8992
PER : Average Reward = 9.9 | Final Loss = 0.7281
```

Visualization Output:



